UNIDAD 4
ARREGLOS Y VECTORES

Por Félix Ferreiras
y Zobiesky Ovalle

Arreglos definidos en tiempo de compilación y vectores

Objetivos

En esta unidad usted aprenderá a:

- Crear, manipular y acceder a los elementos de un arreglo mediante el uso de ciclos
- Usar vectores y entender sus diferencias en comparación con los arreglos de tamaño fijo en tiempo de compilación
- Utilizar los recursos o funciones de los vectores
- Acceder a los elementos de un vector mediante el uso de iteradores
- Trabajar con vectores de estructuras

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Contenido

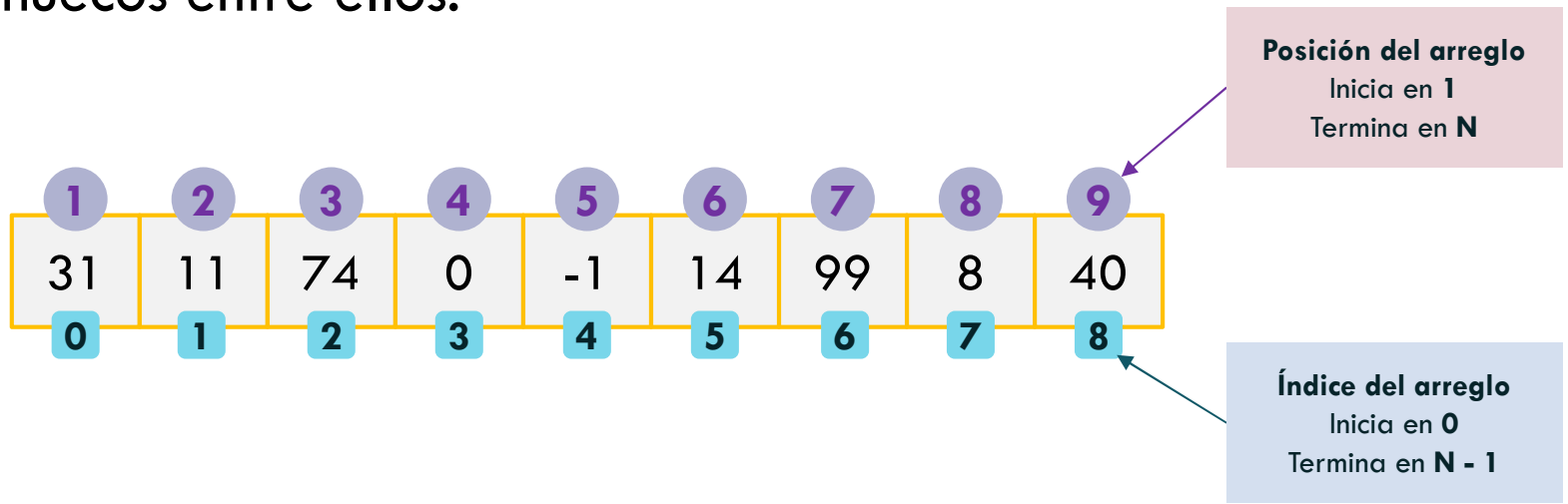
- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

¿Que son los arreglos?

Tipo de variable para almacenar múltiples datos en posiciones contiguas en memoria, bajo un índice.

Arreglos: Concepto

- Es una forma de almacenar información en RAM, como una colección ordenada en secuencia de elementos de datos de un mismo tipo bajo un mismo nombre (identificador).
- Los elementos de un arreglo son almacenados en bloques contiguos dentro de la memoria principal, es decir, no hay huecos entre ellos.



Arreglos: Concepto

- Cada elemento es identificado con una variable sub-indicada, utilizando un sub-índice para referenciar cada elemento de la variable, esto es, un elemento se distingue de otro por medio del sub-índice.
- Las variables sub-indicadas se comportan como las variables simples, esto es, podemos hacer las mismas operaciones (asignación, suma, multiplicación, comparaciones, etc.)
- Cada elemento almacena el mismo **TIPO DE DATO**.
- Cada elemento, en diferentes momentos, puede ir almacenando diferentes valores, del mismo tipo. Podríamos decir que los arreglos nos permiten tratar muchas variables homogéneas (del mismo tipo) con el objetivo de darles el mismo tratamiento (ejemplo: proceso para determinar la moda de un conjunto de valores).

Arreglos: Concepto

- En un arreglo dado puede haber elementos almacenados que tengan el mismo valor pero nunca con el mismo sub-índice
- Los sub-índices se numeran desde cero, esto es: 0, 1, 2, …
- Cada parte de arreglo es un elemento del arreglo.
- Cada elemento dentro del arreglo puede ser accedido utilizando el sub-índice del arreglo.
- Ejemplo sea una variable tipo arreglo que guarde 10 edades de personas. Dicho arreglo lo llamaremos **edad**, con tipo de dato entero (integer) tendríamos algo como esto:
- **edad**

10	25	56	12	42	59	68	26	33	40
0	1	2	3	4	5	6	7	8	9

Arreglos: Concepto

Dónde: **La primera línea** del ejemplo anterior representa los diferentes valores almacenados en el arreglo edad, es decir representa las edades almacenadas.

10	25	56	12	42	59	68	26	33	40

Y la segunda línea, representa el sub-índice por el cual se puede acceder al valor de cada elemento dentro del arreglo.

0	1	2	3	4	5	6	7	8	9

Así para referenciar, usar o manipular la edad con valor de **42,** presentada en el ejemplo anterior, lo hacemos con el nombre del arreglo y entre corchetes el sub-índice que le corresponde, como se muestra a continuación:

edad [4]

De la misma forma si queremos sumar las edades que se encuentra en el sub-índice 3 y el sub-índice 6, lo podríamos hacer de la siguiente forma, asignándole el resultado a una variable numérica llamada sumaEdad:

sumaEdad = edad[3] + edad[6]

Cuyo resultado será sumaEdad = 12 + 68, por tanto sumaEdad = 80.

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Tipos de arreglo

De tamaño fijo, variable y dinámicos

Tipos de arreglos

- De tamaño fijo en tiempo de compilación
- De tamaño fijo en tiempo de ejecución
- De tamaño variable en tiempo de ejecución: Vectores

Arreglos de tamaño fijo: compilación

- Estos arreglos definen su capacidad o tamaño a nivel de compilación, esto quiere decir que es necesario especificar la cantidad de elementos que manejará el arreglo en el código fuente.
- En la unidad anterior manejamos arreglos de este tipo al utilizar variables tipo *char* con más de un caracter, esto es, arreglos de caracteres.

Arreglos de tamaño fijo: compilación

- Declaración de un arreglo de tamaño fijo en tiempo de compilación

```
tipo nombre[tamano];
```

Donde:

tipo es el tipo de dato que manejará el arreglo (ejemplo: *int, bool, char, double)*

nombre es el nombre del arreglo y debe seguir la regla de los identificadores

tamano es un valor entero el cual define la cantidad de elementos que tendrá el arreglo (puede ser un literal, una constante o una variable con valor definido dentro del programa)

Arreglos de tamaño fijo: compilación

- Veamos algunos ejemplos:

```
bool decisiones[10];

float posiciones[3];

const int SIZE = 5;
char vocales[SIZE];

short edades[] = {23, 34, 22, 38, 40};

int capacidades[8] = {100, 80, 250};
```

Declara un arreglo llamado decisiones de 10 elementos de tipo bool

Declara un arreglo llamado posiciones de 3 elementos tipo float

Declara una constante entera llamada SIZE con valor de 5 y declara un arreglo de caracteres con tamaño definido por SIZE, esta forma es la recomendable para definir el tamaño de un arreglo

Declara un arreglo y lo inicializa con 5 elementos, en este caso el tamaño del arreglo queda definido por la cantidad de elementos especificada

Declara un arreglo de enteros de 8 elementos donde las primeras 3 posiciones serán pobladas con los valores 100, 80 y 250

Arreglos de tamaño fijo: compilación

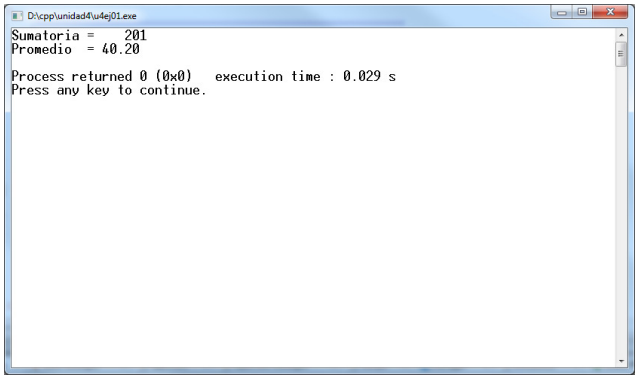
- Veamos un ejemplo:

```
1   #include <iostream>
2   #include <iomanip>
3
4   using namespace std;
5
6   int main(int argc, char** argv)
7   {
8       const int SIZE = 5;
9       int numeros[SIZE] = {34, 23, 33, 66, 45};
10      int sum = 0;
11
12      for (int i = 0; i < SIZE; i++)
13      {
14          sum += numeros[i];
15      }
16
17      cout << "Sumatoria = " << setw(6) << sum << endl;
18      cout << setprecision(2) << fixed;
19      cout << "Promedio  = " << setw(6) << left
20           << (double)sum / SIZE << endl;
21
22      return 0;
23  }
```

En este ejemplo podemos observar que se declara un arreglo cuyo tamaño es definido por la constante SIZE y a su vez es inicializado con 5 valores.

Un ciclo ***for*** accede a todos los elementos del arreglo, desde el primero hasta el último y acumula los valores que en cada posición se encuentran almacenados.

Finalmente imprime el promedio de los valores contenidos en el arreglo. Se utilizan especificadores de formato para la impresión del resultado.



Arreglos de tamaño fijo: ejecución

- Estos arreglos definen su capacidad o tamaño a nivel de ejecución, esto quiere decir la cantidad de elementos que tendrá el arreglo es definido por un valor no determinado en el programa fuente (por ejemplo, solicitud del tamaño por teclado, obtener el tamaño por la evaluación de alguna expresión, el tamaño es retornado como valor al invocar una función, etc.)
- Este tipo de arreglo lo veremos más adelante en la unidad de punteros.

Arreglos de tamaño fijo: ejecución

- Declaración de un arreglo de tamaño fijo en tiempo de ejecución:

```
tipo *nombre = new tipo[tamano];

delete [] nombre;

delete * nombre;
```

Como fue mencionado anteriormente, esto será tratado en detalle en la unidad 6 referente a punteros.

Arreglos de tamaño variable en ejecución <*vector*>:

- Son los arreglos inteligentes o **`<vector>`**, que desarrollaremos en este curso.
- Son arreglos especiales de tamaño variable en tiempo de ejecución, llamados “Vectores” o bien arreglos dinámicos.
- Su tamaño es variable, según se va determinando en la ejecución del programa y atendiendo a las necesidades del programador para resolver determinados problemas.
- Poseen una amplia gama de recursos, con los cuales podemos implementar una serie de operaciones sobre ellos.

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Arreglos de cadenas de caracteres

Cadenas C-estilo

C-Strings

Arreglos de cadena de caracteres

- Son cadenas de tamaño fijo
- Son cadenas c-style (estilo C) las cuales tienen tamaño fijo desde el momento de su creación

```
char nombre[30];
```

La sentencia anterior define una cadena como un arreglo de caracteres

Funciones para trabajar con C-Strings

- Existen diversas funciones incluidas en la librería `<cstring>` que nos facilitan el trabajo para manipular variables de arreglos de ***char*** o C-Strings
- Estas funciones resultarán muy útiles al momento de realizar ciertas tareas que ya se encuentran incluidas en dicha librería en donde para consumirlas bastaría con pasar la data que será trabajada en formato de cadena C-Style

Funciones para trabajar con C-Strings

Función	Descripción
`strlen(str)`	Retorna la longitud de la cadena *str*
`strcat(str1, str2)`	Concatena el contenido de *str2* en *str1*, si *str1* no dispone del espacio disponible para que *str2* pueda ser concatenado en su totalidad, solo se copia lo que cabe en *str1*
`strcpy(str1, str2)`	Copia el contenido de *str2* en *str1*, se sustituye lo que tenga *str1* por el contenido en *str2*
`strncat(str1, str2, n)`	Concatena el contenido de *str2* en *str1*, toma también como argumento la cantidad máxima de elementos (*caracteres*) que pueden ser copiados a *str1* mediante el tercer argumento, *n*
`strncpy(str1, str2, n)`	Copia el contenido de *str2* en *str1*, toma también como argumento la cantidad máxima de elementos (*caracteres*) que pueden ser copiados a *str1* mediante el tercer argumento, *n*
`strcmp(str1, str2)`	Compara *str1* con *str2: r*etorna 0 si str1 es igual a str2; retorna >0 si str1 es mayor que str2 y retorna <0 si str1 es menor que str2
`strstr(str1, str2)`	Retorna un puntero de *char* con la primera ocurrencia de *str2* en *str1* o *nulo si no tiene ocurrencia alguna*

Funciones para trabajar con C-Strings

Función	Descripción
`atoi(str)`	Convierte el valor en *str* a entero (int)
`atol(str)`	Convierte el valor en *str* a entero (long int)
`atof(str)`	Convierte el valor en *str* a un valor de punto flotante (double)
`itoa(valor, str, base)`	Convierte un valor entero a su equivalente numérico de acuerdo a la base especificada y lo almacena como un C-String en *str*. base puede ser: 10 (decimal), 16 (hexadecimal), 8 (octal), 2 (binario) * Esta función no es estándar y puede que algunos compiladores no la hayan implementado

- Estas funciones requieren el uso de la librería **<cstdlib>**

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

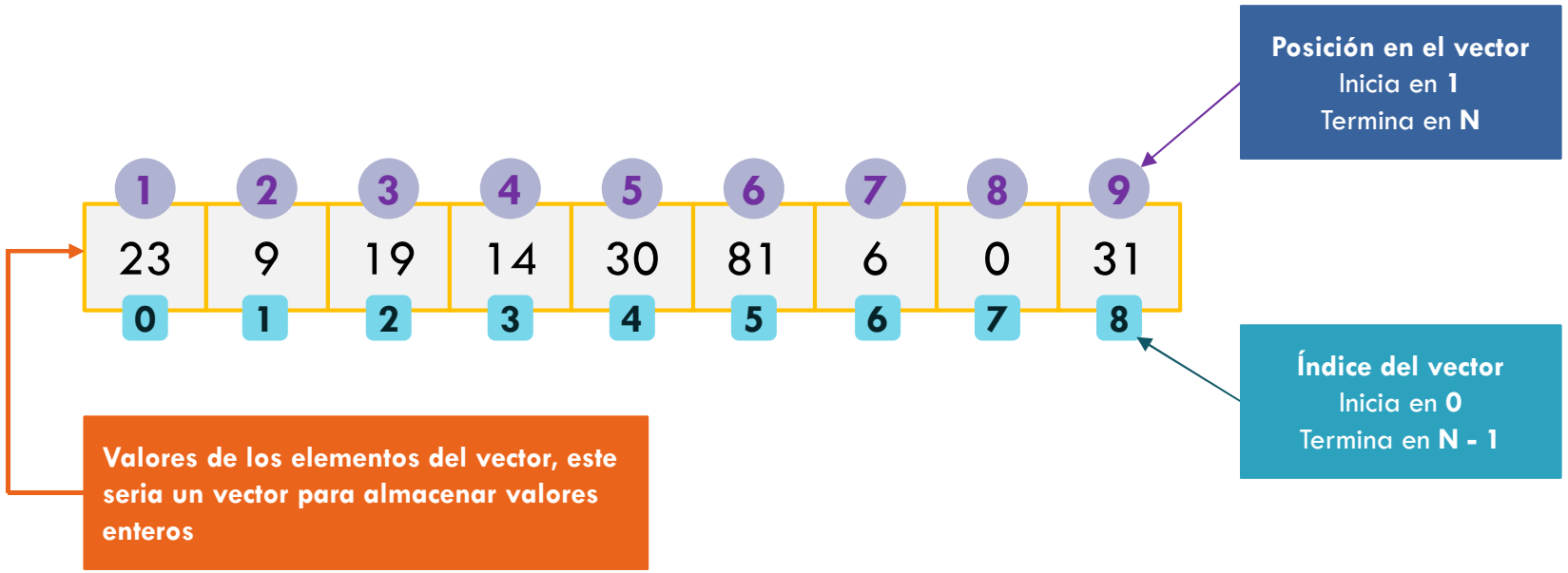
Vectores

Arreglos dinámicos

Vectores: conceptos

- Es un arreglo que puede crecer dinámicamente, esto es, *su tamaño es variable en tiempo de ejecución*
- Es un arreglo inteligente, a diferencia de los arreglos regulares, el almacenamiento es manejado de manera automática, permitiendo ser expandido o contraído a requerimiento en tiempo de ejecución
- Se encuentra implementado en la librería <vector>
- Se requiere incluir dicha librería en el programa:
  - `#include <vector>`

Vectores: conceptos



Vectores: conceptos

- Son más eficientes y seguros accesando a sus elementos por el subíndice.
- Permiten acceso aleatorio de sus elementos en cualquier orden.
- Permiten su redimensionamiento con facilidad.
- Permiten añadir y remover elementos al inicio y después del último.

Vectores: declaración e inicialización

- Declaración de un vector:

```
vector<tipoDato> nombreVector[(tamano[, valorInicial])]
```

- Donde:
  - **tipoDato:** representa el tipo de datos que manejara el vector (ejemplo: double, int, float, bool, string, una estructura, etc.)
  - **nombreVector:** es el nombre que llevara el vector, el cual debe estar sujeto a las reglas de los identificadores
  - **tamano:** (opcional) es un entero el cual define una cantidad inicial de elementos
  - **valorInicial:** (opcional) representa el valor que tendrá la cantidad de elementos especificados por ***tamano*** y debe ser una variable, constante, literal o expresión que al ser evaluada de un valor del mismo tipo que ***tipoDato***

Vectores: declaración e inicialización

- Veamos algunos ejemplos:

```
vector<string> nombres;
```

Declara un vector de string llamado nombres

```
vector<double> sueldos;
```

Declara un vector para manejar valores double llamado sueldo

```
vector<int> numeros(10);
```

Declara un vector tipo int llamado numeros con 10 elementos como cantidad inicial (puede crecer o decrecer al igual que cualquier otro vector)

```
vector<bool> procesados(20, false);
```

Declara un vector para manejar valores booleanos con una cantidad inicial de 20 elementos los cuales todos serán *false*

```
vector<Coordenadas> coordenadas;
```

Declara un vector de estructura llamado coordenadas el cual podrá manipular elementos tipo Coordenadas la cual es una estructura

Contenido

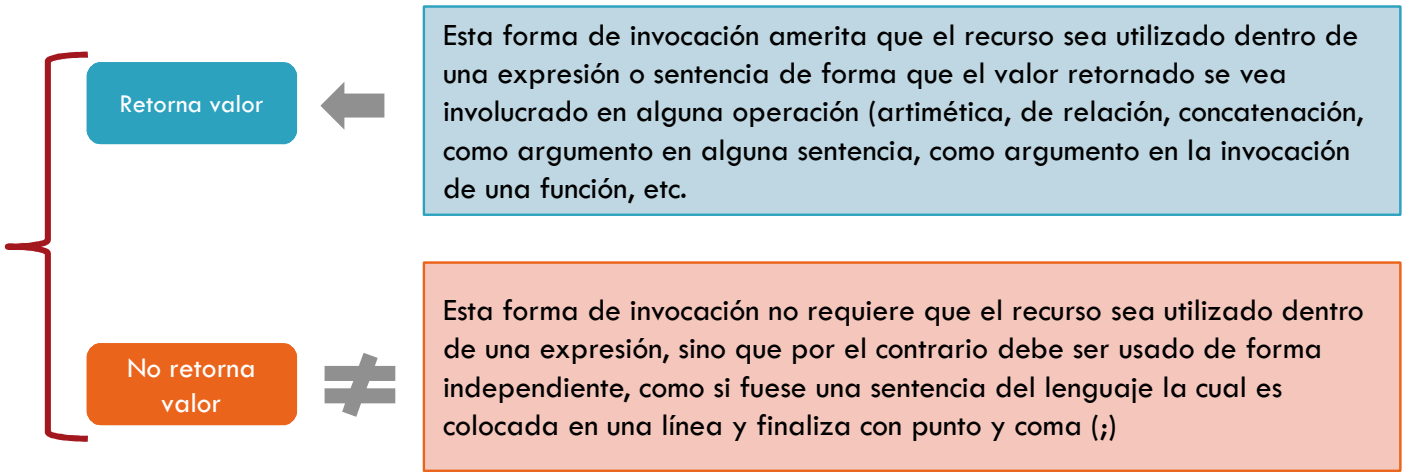
- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Recursos de vectores

Funciones en vectores

Recursos de vectores

- Los recursos de vectores nos brindan diversas funcionalidades que facilitan muchas tareas al programador
- Existen diferentes tipos de recursos o funciones de vectores, es importante conocer algunos detalles sobre su forma de uso o invocación (al igual que con los recursos de string):



Recursos de vectores

Recurso	Descripción
`at(i)`	Permite acceder al elemento ubicado en el índice *i*
`size()`	Retorna el número de elementos que contiene el vector
`push_back(valor)`	Permite agregar un nuevo elemento al vector al final del mismo
`pop_back()`	Remueve el ultimo elemento del vector
`empty()`	Retorna *1 (true)* si el vector está vacío, de lo contrario, si tiene al menos un elemento, retorna *cero (false)*
`erase(iter)` `erase(iter1, iter2)`	Remueve el elemento indicado por el iterador *iter* Remueve los elementos en el rango de los iteradores *iter1* e *iter2*
`swap(vector)`	Permite intercambiar el valor dos variables tipo vector, esto es, que los elementos de un vector pasarían al especificado entre paréntesis y los elementos del vector entre paréntesis pasarían al vector que invoca el recurso
`clear()`	Remueve todos los elementos del vector dejándolo vacío
`insert(iter, valor)`	Inserta el valor especificado a partir de la posición indicada por el iterador *iter* en el vector

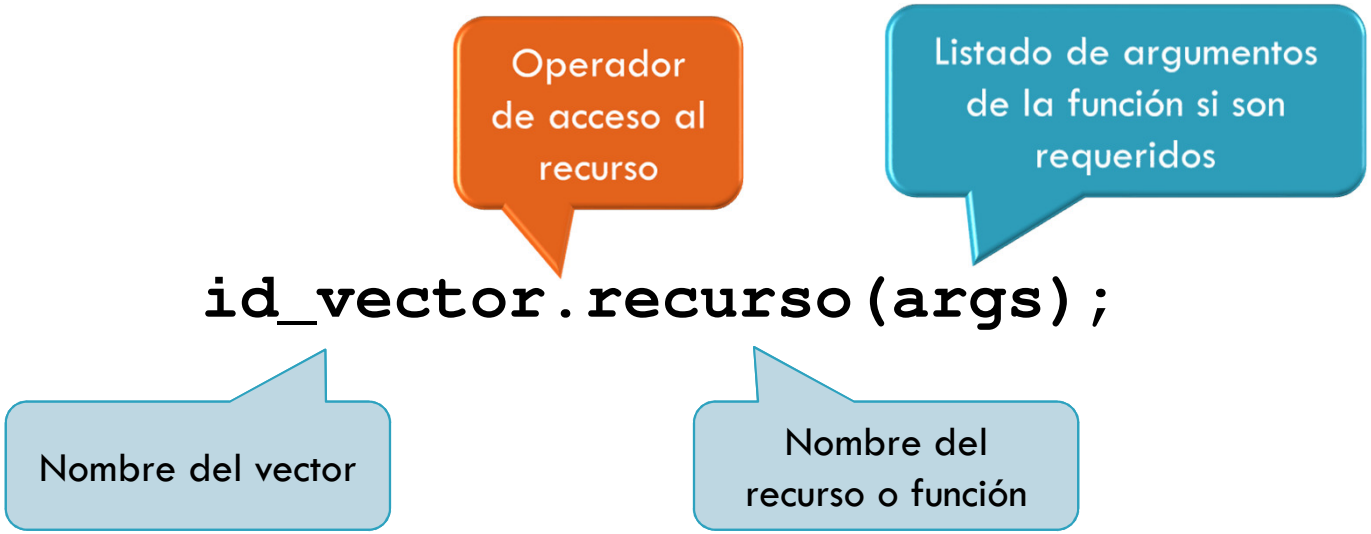
Recursos de vectores

Recurso	Descripción
`begin()`	Retorna el iterador que se encuentra al inicio del vector, esto seria la primera posición
`end()`	Retorna el iterador que se encuentra una posición mas al final del vector, esto seria la posición N + 1
`front()`	Retorna el primer elemento del vector
`back()`	Retorna el ultimo elemento del vector
`resize(n)`	Permite redimensionar el vector asignando un nuevo tamano, en caso de que n sea menor al tamano del vector entonces vector es truncado hasta n elementos

- Es importante notar que existen más funciones o recursos de vector y que los aquí mencionados tienen implementaciones adicionales las cuales agregan otras formas de uso. Les invitamos a visitar http://cplusplus.com donde podrá encontrar una rica referencia sobre la librería vector y sus funciones.
- Exhortamos a que examine los ejemplos que se encuentran como parte del material de apoyo de esta unidad donde podrá encontrar diversos ejemplos de uso de recursos de vector, modifique el código a su consideración y juegue con ellos implementando diferente variantes.

¿Cómo invocamos un recurso?

- Para invocar una función o recurso de vector utilizamos el punto para separar la variable vector de la función



Operación de asignación

- Puede utilizar el operador de asignación con vectores, esto es, que a una variable tipo vector se le puede asignar el valor de otro vector sustituyendo todos sus elementos por los elementos del vector de la derecha en la operación.
- Los dos vectores deben ser del mismo tipo.

```
vector1 = vector2;
```

Operadores de vectores

Clasificación	Operador	Comentarios
Asignación	=	Asigna un vector a otro vector
Comparación	>	Mayor que
	>=	Mayor o igual que
	<	Menor que
	<=	Menor o igual que
	==	Igual que
	!=	Distinto de
Referencia	[i]	Permite obtener el valor de la posición especificada en i

- Los operadores de relacion con vectores comparan secuencialmente de forma reciproca cada elemento o posición del vector contra el otro vector, deteniéndose donde se encuentre una diferencia y comparando el valor en dicha posición, de tal forma se obtiene el resultado final de la operación de relación.

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Iteradores

Posiciones en un vector

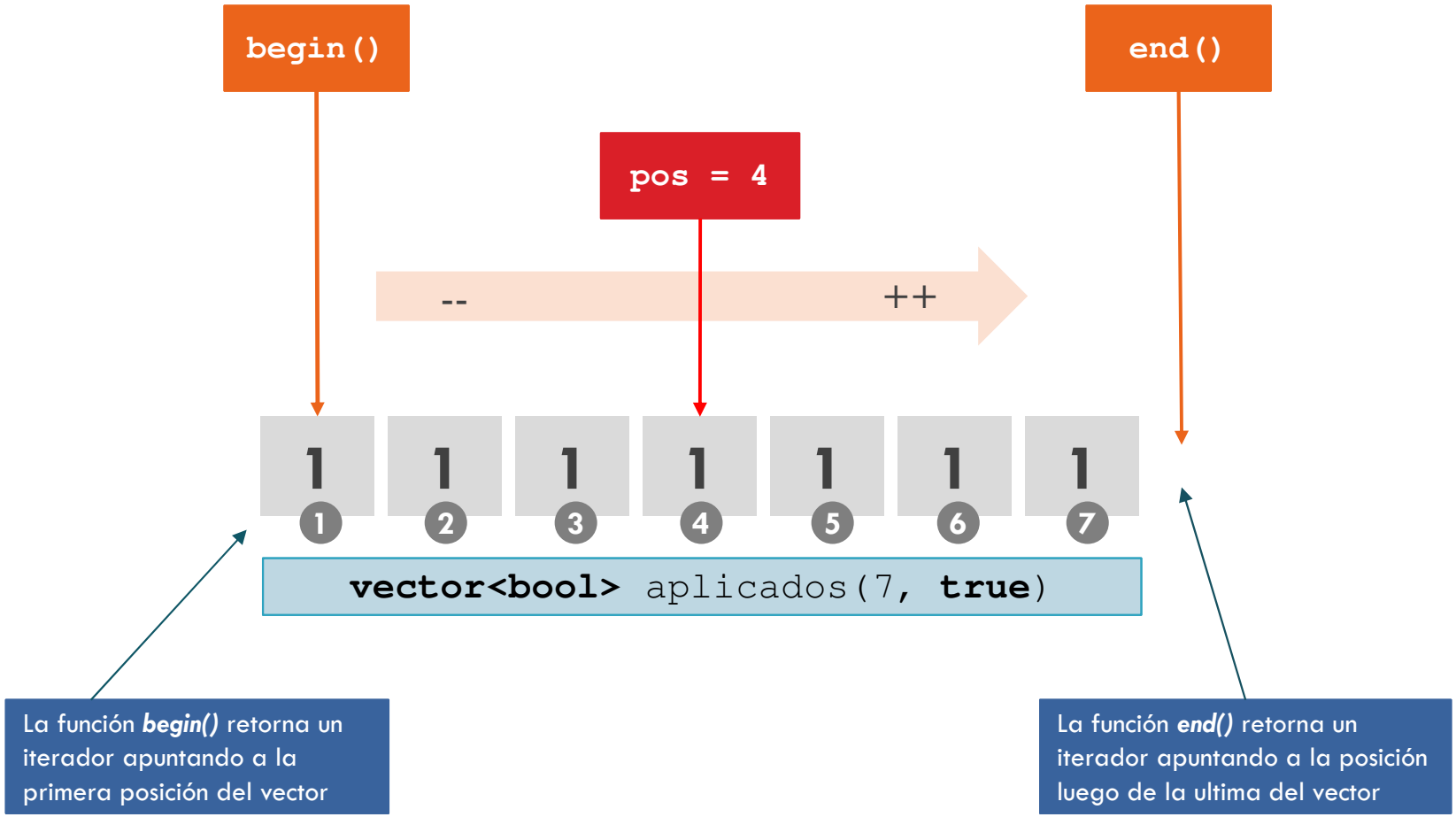
Iteradores

- En C++, un iterador de un `<vector>` es un medio para acceder a los elementos del vector, uno a la vez.
- Para declarar un iterador de `<vector>` la sintaxis es:

```
vector::iterator id;
```

- ***id*** es un identificador cualquiera (el nombre del iterador)
- Al momento de hacer la declaración de un iterador lo podemos poner a apuntar al primer elemento o a la posición que esta un elemento después del último en el vector.

Iteradores


begin()
end()
pos = 4
--
++
1 1 1 1 1 1 1
1 2 3 4 5 6 7
vector<bool> aplicados(7, true)
La función begin() retorna un iterador apuntando a la primera posición del vector
La función end() retorna un iterador apuntando a la posición luego de la ultima del vector

Iteradores

- Como declarar e inicializar un iterador

```
vector<int> val(10, 0);
```

Tomemos el vector ***valores*** con el cual queremos utilizar un iterador para apuntar una posición dentro del vector

```
vector::iterator varIter;
varIter = val.begin();
```

Con la primera línea declaramos un iterador llamado ***varIter*** y con la segunda línea le damos el valor de la posición uno del vector ***val*** . Note que debe existir previamente una variable ***vector*** con algún valor para que nuestro iterador pueda ser utilizado como queremos. Si el vector solo fue declarada pero no tiene elementos, entonces nuestro iterador no tendrá una posición correcta. Recuerde que la función ***begin*** retorna un iterador con la primera posición del ***vector***.

```
vector::iterator varIter = val.begin();
```

Otra forma es declara e inicializar el iterador en una misma línea de código. Esto hace exactamente lo mismo a las dos líneas anteriores.

Operaciones con los iteradores

- Movernos de posición operando aritméticamente el iterador sumando y restando
- Siendo

```
vector<double> sueldos;
sueldos.push_back(45000.0);
sueldos.push_back(34800.0);
sueldos.push_back(87000.0);
sueldos.push_back(17550.0);
vector::iterator ite = sueldos.begin();
```

Operación	Resultado	Posición / Elemento
`ite + 0`	Apunta al 1er elemento	`1 = 45000.0`
`ite + 2`	Apunta al 3er elemento	`3 = 87000.0`
`ite + 3`	Apunta al 4to elemento	`4 = 17500.0`

Operaciones con los iteradores

- Movernos de posición operando aritméticamente el iterador sumando y restando
- Siendo

```
vector<double> sueldos;
sueldos.push_back(45000.0);
sueldos.push_back(34800.0);
sueldos.push_back(87000.0);
sueldos.push_back(17550.0);
vector::iterator ite = sueldos.end();
```

Operación	Resultado	Posición / Elemento
`ite - 1`	Apunta al ultimo elemento	`4 = 17550.0`
`ite - 3`	Apunta al 2do elemento	`2 = 34800.0`
`ite - 4`	Apunta al 1er elemento	`1 = 45000.0`

Accediendo mediante iteradores

- Para acceder el valor almacenado por el elemento apuntado por el iterador, se realiza la operación llamada indireccionamiento, la cual se representa por el asterisco:
- Siendo

```
vector<double> sueldos;
sueldos.push_back(45000.0);
sueldos.push_back(34800.0);
sueldos.push_back(87000.0);
sueldos.push_back(17550.0);
vector::iterator ite = sueldos.begin();
```

Sentencia	Resultado
`cout << *ite;`	45000.0
`cout << *(ite + 2);`	87000.0
`cout << *(ite + 1);`	34800.0

Iteradores

- Los iteradores tienen un tipo, esto es, apuntan a un tipo de dato, el tipo de dato almacenado por la colección y como tal hay que declararlos
- Con el uso de iteradores, el acceso a los elementos individuales del vector es seguro por que tienen la inteligencia construida para avisar cuando se intenta acceder a una posición vacía en dicha cadena (esto último es parte del manejo de excepciones)

Iteradores

- Otra forma para acceder elementos en un vector:
  - Usando el recurso at (indice)
  - Usando el operador [indice]

Les invitamos a examinar y jugar con los ejercicios de ejemplo que son provistos para cada unidad en el material de este curso.

Contenido

- ¿Qué son los arreglos?
- Tipos de arreglos
- Arreglos de cadena de caracteres
- Vectores
- Recursos en vectores
- Iteradores
- Vectores de estructuras

Vectores de estructuras

vector<Estructura>

Vectores de estructura

- Los vectores pueden contener también tipos de datos compuestos como es el caso de las estructuras
- Al definir el vector solo es necesario colocar el nombre de la estructura en el lugar donde colocaríamos el tipo de dato
- Para agregar una estructura completa (con todos sus miembros) debemos utilizar la función ***push_back()*** y pasar como argumento el nombre de la variable tipo estructura que deseamos adicionar al vector, hay que tomar en cuenta que esa variable debe ser del mismo tipo de estructura que la especificada al momento de declarar el vector

Vectores de estructura

```cpp
#include <iostream>
#include <vector>

using namespace std;

struct Triangulo
{
    double base;
    double altura;
};

int main(int argc, char** argv)
{
    Triangulo tri;
    vector<Triangulo> triangulos;

    do
    {
        cout << "Base   : ";
        cin >> tri.base;
        if (tri.base == 0) break;
        cout << "Altura: ";
        cin >> tri.altura;

        triangulos.push_back(tri);
    } while (true);

    return 0;
}
```

Agregando una estructura al vector

En este ejemplo definimos una variable tipo Triangulo llamada *tri* la cual será responsable de almacenar cada miembro y posteriormente agregarla al vector.

Con la operación *push_back()* lo que debe recibir el vector es una variable tipo Triangulo quedando como responsabilidad del programador garantizar que dicha variable tenga todos sus miembros con la data que deben correspondiente.

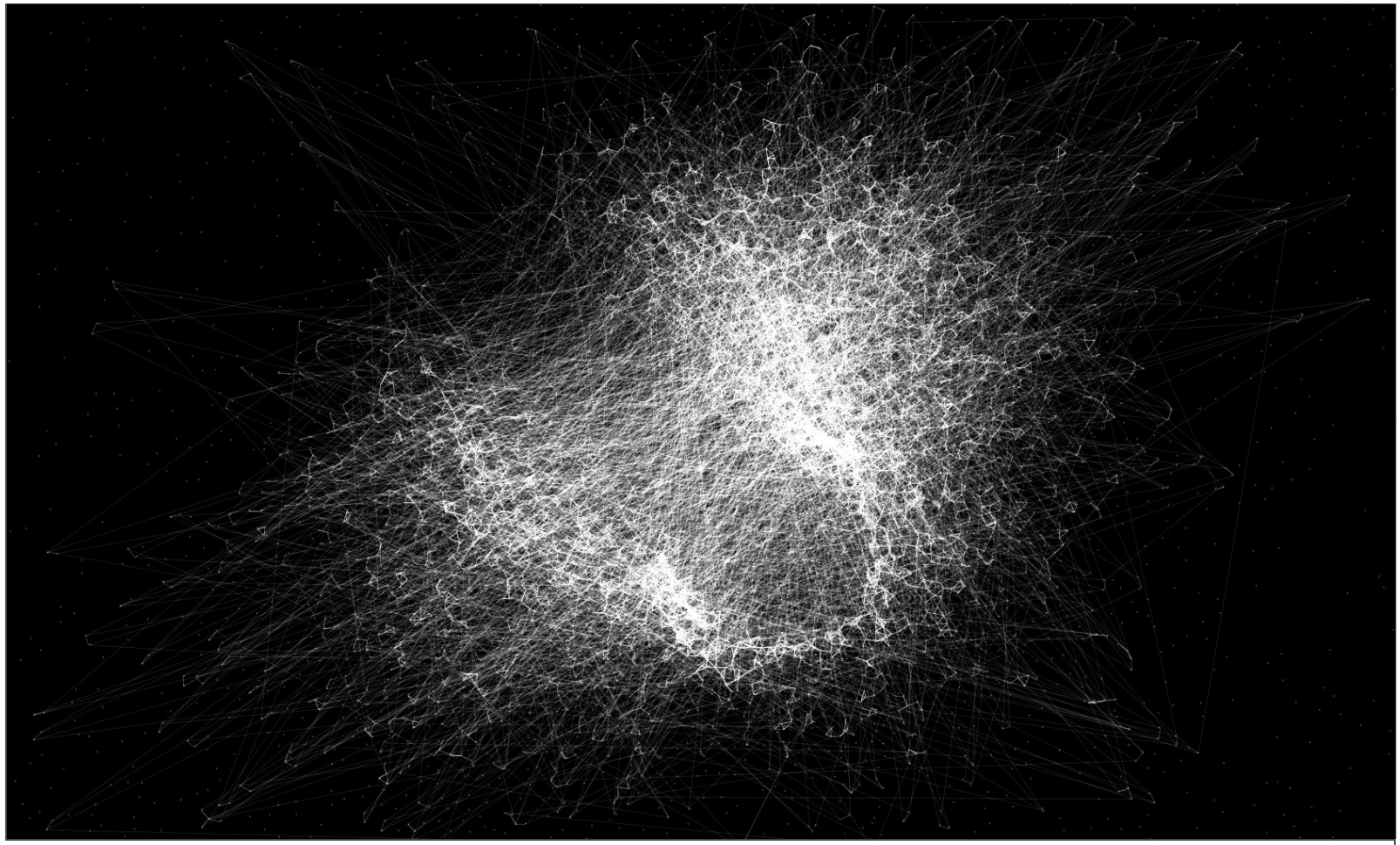
De esta forma podemos crear un grupo de triángulos o de cualquier tipo de estructura.

Resumen

- En esta unidad hemos aprendido:
  - Los ***vectores*** son arreglos inteligentes que contienen diversos recursos o funciones que nos facilitan su manipulación
  - Los ***vectores*** pueden ser manejadas mediante el uso de iteradores permitiéndonos apuntar a sus elementos posición por posición
  - Existen arreglos de tamano fijo en tiempo de compilacion, arreglos de tamano fijo en tiempo de ejecucion (ver punteros) y arreglos de tamano variable en tiempo de ejecucion, los vectores
  - Usualmente el tipo de sentencia utilizada para procesar los arreglos y vectores son las sententencias de ciclo, siendo la sentencia ***for*** la mas comun para esta labor

Bibliografía

- Addison Wesley, Tony Gaddis. (2009). Starting Out with C++ Brief: From Control Structures through Objects. San Francisco, California. 6th Edition.
- Aprende a programar en C++. (2010). Página dedicada a la enseñanza de C++ con libro de libre descarga, recuperado el 3 Junio 2013 de: http://c.conclase.net/curso/index.php
- Página dedicada al lenguaje de programación C++ (2000 – 2014). Tutoriales y recursos de libre descarga, recuperado Enero 2014 en: http://www.cplusplus.com All rights reserved - v3.1
- Paul Deitel, Harvey Deitel (2011). C++ How to Program, New York, 8th Edition, Prentice Hall.



Unidad 4: Arreglos y vectores

Por Félix Ferreiras y Zobiesky Ovalle